# FYBCA Semester 2 — Complete Exam Notes

## Subject 1: Object Oriented Programming in C++

KBCNMU | CA-OOP | Exam: 30 Marks | Complete Notes

# TOPIC 1: OOP Concepts — Object, Class, Encapsulation, Abstraction

## 1.1 Simple Explanation (Beginner Friendly)

OOP stands for Object-Oriented Programming. Instead of writing code as a list of instructions, OOP organizes code around OBJECTS — just like real-world entities.

Think of it this way: A CAR is an object. It has properties: color, brand, speed (called ATTRIBUTES). It has behaviors: start, stop, accelerate (called METHODS). In C++, we use CLASSES to define objects. A class is like a blueprint, and an object is the actual product made from that blueprint.

## 1.2 Detailed Explanation (College Level)

### What is a Class?

A CLASS is a user-defined data type that bundles data (variables) and functions (methods) together. It acts as a template for creating objects.

```
class Car {
public:
    string brand;    // attribute
    int speed;       // attribute
    void start() {   // method
        cout << "Car started";
    }
};
```

### What is an Object?

An OBJECT is an instance of a class. When a class is defined, no memory is allocated. Memory is allocated only when an object is created.

```
Car myCar;               // Object creation
myCar.brand = "Toyota";
myCar.speed = 120;
myCar.start();           // Output: Car started
```

### Encapsulation

ENCAPSULATION means binding data and functions together inside a class and restricting direct access to some components. It is achieved using access specifiers: private, public, protected.

```
class BankAccount {
private:
    float balance;    // hidden from outside
public:
    void deposit(float amount) { balance += amount; }
    float getBalance() { return balance; }
};
```

- Data hiding is achieved by making variables private.
- Only public methods can access private data.
- This protects data from accidental modification.

### Abstraction

ABSTRACTION means showing only relevant features and hiding unnecessary implementation details. It focuses on WHAT an object does rather than HOW it does it.

Example: When you press the accelerator in a car, you just press the pedal. You don't need to know how the engine works internally.

- Abstraction is implemented using Abstract Classes.
- In C++, abstraction is achieved using pure virtual functions.

## 1.3 Key Definitions (Exam-Ready)

| Term | Definition |
|---|---|
| Class | User-defined data type containing data members (variables) and member functions (methods). |
| Object | An instance of a class. Occupies memory and has state, behavior, and identity. |
| Encapsulation | Wrapping data and functions into a single unit (class) and hiding the data. |
| Abstraction | Hiding implementation details and showing only essential features to the user. |
| Data Hiding | Preventing external access to class data using private access specifier. |
| Access Specifier | Keywords that control access to class members: public, private, protected. |

## 1.4 Four Pillars of OOP

| Pillar | Definition | Achieved By |
|---|---|---|
| Encapsulation | Wrapping data + functions in a class | Access specifiers (private/public) |
| Abstraction | Hiding unnecessary details | Abstract class, pure virtual functions |
| Inheritance | One class acquires properties of another | Derived class : base class |
| Polymorphism | Same name, different behavior | Function/operator overloading, virtual functions |

## 1.5 Differences: Encapsulation vs Abstraction

| Encapsulation | Abstraction |
|---|---|
| Hides DATA from outside world | Hides IMPLEMENTATION details |
| Achieved by access specifiers | Achieved by abstract classes |
| Protects data integrity | Reduces programming complexity |
| Binds data + functions in class | Shows only relevant features |

| Class | Object |
|---|---|
| Blueprint or template | Real entity created from the class |
| Does NOT occupy memory | Occupies memory at runtime |
| Defined using 'class' keyword | Created by declaring variable of class type |

| | |
|---|---|
| One class can exist | Multiple objects created from one class |

## 1.6 Frequently Asked Exam Questions

**Important Questions (Must Prepare)**

- Define OOP. What are its features? (2 marks)
- What is a class and object? Give an example. (4 marks)
- What is encapsulation? How is it achieved in C++? (4 marks)
- Differentiate between abstraction and encapsulation. (4 marks)
- List and explain the four pillars of OOP. (6 marks)
- What are access specifiers in C++? Explain with example. (4 marks)

## 1.7 Important Points to Remember

**Must Remember for Exam**

- C++ supports 4 main OOP pillars: Encapsulation, Abstraction, Inheritance, Polymorphism.
- A class is a blueprint; an object is the real entity created from that blueprint.
- Private members are accessible ONLY inside the class.
- Public members are accessible from anywhere in the program.
- Abstraction hides complexity; Encapsulation hides data.
- C++ is partially object-oriented (supports procedural programming too).

## 1.8 Keywords for Exam

class, object, instance, data member, member function, access specifier, private, public, protected, encapsulation, abstraction, data hiding, blueprint, OOP pillars

# TOPIC 2: Constructors and Destructor

## 2.1 Simple Explanation

When you create an object, you often need to give it initial values. CONSTRUCTORS automatically initialize objects when they are created. DESTRUCTORS automatically clean up when objects are destroyed.

Think of a Constructor like a welcome ceremony when a guest arrives, and a Destructor like a farewell when they leave.

## 2.2 What is a Constructor?

A constructor is a special member function that has the SAME NAME as the class and has NO return type (not even void). It is automatically called when an object is created.

```
class Student {
public:
    string name;
    int age;
    Student(string n, int a) {    // Constructor
        name = n;
        age = a;
        cout << "Student created: " << name;
    }
};

Student s1("Rahul", 20);  // Constructor called automatically
```

## 2.3 Types of Constructors

### 1. Default Constructor

Takes no parameters. Called automatically when object is created without arguments.

```
class Student {
public:
    string name;
    Student() {     // Default constructor
        name = "Unknown";
        cout << "Default constructor called";
    }
};
Student s1;   // Calls default constructor
```

### 2. Parameterized Constructor

Takes parameters to initialize object with specific values.

```
class Student {
public:
    string name;
    int age;
    Student(string n, int a) {    // Parameterized constructor
        name = n;
        age = a;
    }
};
Student s1("Rahul", 20);   // Calls parameterized constructor
```

### 3. Copy Constructor

Creates a new object as a copy of an existing object.

```
class Student {
public:
    string name;
    Student(Student &s) {   // Copy constructor
        name = s.name;
    }
};
Student s1("Rahul");
Student s2(s1);   // Copies s1 into s2
```

### 4. Constructor Overloading

A class can have multiple constructors with different parameter lists. The compiler automatically selects the correct one.

```
class Box {
public:
    int l, w, h;
    Box() { l=w=h=1; }                      // Default
    Box(int x) { l=w=h=x; }                 // One param
    Box(int a, int b, int c) { l=a; w=b; h=c; }  // Three params
};
```

## 2.4 Destructor

A destructor is called automatically when an object goes out of scope or is deleted. It has the SAME NAME as the class preceded by TILDE (~). No parameters, no return type.

```
class Student {
public:
    Student()  { cout << "Constructor called"; }
    ~Student() { cout << "Destructor called"; }  // Note tilde ~
};
// When Student object is destroyed, destructor is called automatically
```

## 2.5 Key Definitions

| Term | Definition |
|---|---|
| Constructor | Special member function with same name as class, no return type, auto-called when object is created. |
| Default Constructor | Constructor with no parameters. Provides default initialization. |
| Parameterized Constructor | Constructor with parameters to initialize object with specific values. |
| Copy Constructor | Constructor that creates a new object as a copy of another object. |
| Destructor | Special function with ~ prefix, same name as class, auto-called when object is destroyed. |

## 2.6 Comparison: Constructor vs Destructor

| Feature | Constructor | Destructor |
|---|---|---|
| When called | Object is CREATED | Object is DESTROYED |

| | | |
|---|---|---|
| Overloading | Can be overloaded (multiple) | Cannot be overloaded (only one) |
| Parameters | Can have parameters | Cannot have parameters |
| Return type | No return type | No return type |
| Name | Same as class | ~className |
| Purpose | Initialize data | Release resources |

## 2.7 Important Points

**Must Remember**

- Constructor name is SAME as class name.
- Constructor has NO return type (not even void).
- Constructor is called AUTOMATICALLY when object is created.
- Destructor uses TILDE (~) before class name.
- A class can have MULTIPLE constructors (overloading).
- A class can have ONLY ONE destructor.
- If you don't define a constructor, compiler provides a default one.
- Destructor is useful for releasing dynamic memory (delete, free).

## 2.8 Exam Questions

**Questions to Prepare**

- What is a constructor? Explain its types with examples. (6 marks)
- Differentiate between constructor and destructor. (4 marks)
- Write a C++ program demonstrating default, parameterized, and copy constructors. (6 marks)
- What is constructor overloading? Give example. (4 marks)
- When is a destructor called? (2 marks)

## 2.9 Keywords

constructor, destructor, default constructor, parameterized constructor, copy constructor, constructor overloading, tilde (~), initialization, object creation, object destruction

# TOPIC 3: Friend Function and Friend Class

## 3.1 Simple Explanation

Normally, private members of a class cannot be accessed from outside. But sometimes a non-member function needs access to private data. For this, C++ provides the FRIEND concept. A friend function is like a trusted person given special permission to enter private areas.

## 3.2 Friend Function

A friend function is declared inside the class using 'friend' keyword but defined OUTSIDE the class. It can access all private and protected members of the class.

```
class Distance {
private:
    int meters;
public:
    Distance(int m) { meters = m; }
    friend void showDistance(Distance d);   // Declaration with 'friend'
};

void showDistance(Distance d) {    // Definition outside class
    cout << "Distance: " << d.meters;   // Accessing private member
}

int main() {
    Distance d1(50);
    showDistance(d1);   // Output: Distance: 50
}
```

## 3.3 Properties of Friend Function

- Declared with 'friend' keyword inside the class.
- Defined outside the class without any scope operator.
- NOT a member of the class.
- Can access private and protected members.
- Can be friend to more than one class simultaneously.
- Does NOT have 'this' pointer.

## 3.4 Friend Class

When an entire class is declared as friend, ALL member functions of that class can access private and protected members of another class.

```
class Box {
private:
    int length;
public:
    Box(int l) { length = l; }
    friend class Display;   // Display class is friend of Box
};

class Display {
public:
    void show(Box b) {
        cout << b.length;   // Can access Box's private member
    }
};
```

## 3.5 Key Definitions

| Term | Definition |
|---|---|
| Friend Function | Non-member function with access to private and protected members, declared using 'friend' keyword. |
| Friend Class | A class whose all member functions can access private and protected members of another class. |
| this pointer | Pointer available in member functions pointing to the current object. Friend functions do NOT have this. |

## 3.6 Comparison: Friend Function vs Member Function

| Friend Function | Member Function |
|---|---|
| Not a member of the class | Is a member of the class |
| Declared with 'friend' keyword | No special keyword needed |
| No 'this' pointer | Has 'this' pointer |
| Called without object: func(obj) | Called with object: obj.func() |
| Can be friend to multiple classes | Belongs to only one class |

## 3.7 Important Points

**Must Remember**

- Friend function is declared INSIDE the class but defined OUTSIDE.
- Friendship is NOT mutual: if A is friend of B, B is not auto-friend of A.
- Friendship is NOT inherited by derived classes.
- Friend function can be friend of multiple classes at the same time.
- Friend function does NOT have 'this' pointer.
- Using too many friend functions violates data hiding principle.

## 3.8 Exam Questions

**Questions to Prepare**

- What is a friend function? Explain with an example. (4 marks)
- What is a friend class? Write a program to demonstrate it. (4 marks)
- Write properties of friend functions. (4 marks)
- Differentiate between friend function and member function. (4 marks)

# TOPIC 4: Inheritance — Types, Visibility Modes, Virtual Base Class

## 4.1 Simple Explanation

Inheritance is the ability of a class to acquire (inherit) properties and behaviors of another class. The class that gives properties is called PARENT (Base) class, and the class that receives is called CHILD (Derived) class. Just like a child inherits features from parents, a derived class inherits members from a base class.

## 4.2 Syntax of Inheritance

```
class DerivedClass : accessSpecifier BaseClass {
    // Additional members
};

// Example:
class Animal {
public:
    void eat() { cout << "Eating"; }
};

class Dog : public Animal {   // Dog inherits Animal
public:
    void bark() { cout << "Barking"; }
};

Dog d;
d.eat();    // Inherited from Animal
d.bark();   // Dog's own method
```

## 4.3 Types of Inheritance

| Type | Description | Syntax |
|---|---|---|
| Single | One base class, one derived class | class B : public A {} |
| Multiple | Multiple base classes, one derived | class C : public A, public B {} |
| Multilevel | Chain: A inherits to B, B inherits to C | class C : public B {} (B already inherits A) |
| Hierarchical | One base, multiple derived classes | class B:A {} and class C:A {} |
| Hybrid | Combination of two or more types | Mix of above types |

## 4.4 Visibility Modes in Inheritance

| Base Class Member | Public Inheritance | Protected Inheritance | Private Inheritance |
|---|---|---|---|
| public member | public in derived | protected in derived | private in derived |
| protected member | protected in derived | protected in derived | private in derived |
| private member | NOT ACCESSIBLE | NOT ACCESSIBLE | NOT ACCESSIBLE |

KEY RULE: Private members of base class are NEVER directly accessible in derived class, regardless of inheritance mode.

## 4.5 Virtual Base Class — Diamond Problem

In multiple inheritance, if two classes B and C both inherit from A, and D inherits from both B and C — then D gets TWO copies of A. This creates ambiguity called the DIAMOND PROBLEM.

Solution: Use VIRTUAL inheritance — ensures only ONE copy of the base class exists.

```
class A {
public:
    int x;
};
class B : virtual public A {};   // virtual keyword
class C : virtual public A {};   // virtual keyword
class D : public B, public C {}; // Only ONE copy of A in D

D obj;
obj.x = 10;   // No ambiguity now
```

## 4.6 Key Definitions

| Term | Definition |
|---|---|
| Inheritance | Mechanism by which one class acquires properties and behavior of another class. |
| Base Class | The class from which properties are inherited. Also called parent/super class. |
| Derived Class | The class that inherits properties. Also called child/sub class. |
| Multiple Inheritance | A derived class inheriting from more than one base class. |
| Virtual Base Class | Base class declared with 'virtual' to prevent duplicate copies in diamond inheritance. |
| Diamond Problem | Ambiguity when a class inherits from two classes that share a common base class. |

## 4.7 Important Points

**Must Remember**

- private members of base class are NEVER accessible in derived class.
- protected members become accessible in derived class but not outside.
- Public inheritance represents IS-A relationship (Dog IS-A Animal).
- Virtual base class solves the diamond problem.
- Constructor of base class is called BEFORE derived class constructor.
- Destructor of derived class is called BEFORE base class destructor (REVERSE order).
- C++ supports all 5 types of inheritance.

## 4.8 Exam Questions

**Questions to Prepare**

- What is inheritance? Explain its types with examples. (6 marks)
- Explain visibility modes in inheritance with table. (4 marks)

- What is diamond problem? How is it solved using virtual base class? (6 marks)
- Differentiate between single and multiple inheritance. (4 marks)
- Write a program demonstrating multilevel inheritance. (6 marks)

# TOPIC 5: Virtual Functions, Pure Virtual Functions & Abstract Class

## 5.1 Simple Explanation

Imagine base class Animal has a function speak(). Dog should say 'Woof' and Cat should say 'Meow'. With normal functions, the base class version is always called through a base pointer. With VIRTUAL functions, the correct derived class version is called — this is RUNTIME POLYMORPHISM.

## 5.2 The Problem Without Virtual Functions

```
class Animal {
public:
    void speak() { cout << "Animal sound"; }   // normal function
};
class Dog : public Animal {
public:
    void speak() { cout << "Woof"; }
};

Animal *ptr = new Dog();
ptr->speak();   // Output: "Animal sound"  <-- WRONG!
```

The base class version is called — not the derived version. This is Static/Early Binding.

## 5.3 Virtual Functions — The Solution

```
class Animal {
public:
    virtual void speak() { cout << "Animal sound"; }   // virtual keyword
};
class Dog : public Animal {
public:
    void speak() { cout << "Woof"; }   // overrides virtual
};

Animal *ptr = new Dog();
ptr->speak();   // Output: "Woof"  <-- CORRECT!
```

Now the derived class function is called. This is DYNAMIC BINDING (Runtime Polymorphism). The compiler uses a VTABLE (virtual function table) internally.

## 5.4 Pure Virtual Function

A pure virtual function has NO definition in the base class. It forces derived classes to implement their own version. Syntax: virtual returnType funcName() = 0;

```
class Shape {
public:
    virtual float area() = 0;   // Pure virtual (= 0 means no body)
};

class Circle : public Shape {
    float r;
public:
    Circle(float radius) { r = radius; }
    float area() { return 3.14 * r * r; }   // MUST implement
};
```

```
class Rectangle : public Shape {
    float l, w;
public:
    Rectangle(float len, float wid) { l=len; w=wid; }
    float area() { return l * w; }   // MUST implement
};
```

## 5.5 Abstract Class

A class with AT LEAST ONE pure virtual function is called an ABSTRACT CLASS. Objects of abstract class CANNOT be created directly. It serves as a base template only.

```
Shape s;     // ERROR: Cannot create object of abstract class
Circle c(5); // OK: Circle implements all pure virtual functions
```

- Abstract class CAN have constructors.
- Abstract class CAN have concrete (regular) member functions.
- If derived class doesn't implement all pure virtual functions, it also becomes abstract.

## 5.6 Key Definitions

| Term | Definition |
|---|---|
| Virtual Function | Function declared with 'virtual' in base class that enables runtime polymorphism. |
| Pure Virtual Function | Virtual function with no body, declared with '=0'. Forces derived classes to implement it. |
| Abstract Class | Class with at least one pure virtual function. Cannot be instantiated directly. |
| Dynamic Binding | Selecting the correct function at runtime based on actual object type. Also called late binding. |
| Static Binding | Function calls resolved at compile time. Also called early binding. |
| vtable | Virtual function table maintained by compiler to track which virtual function to call. |

## 5.7 Comparison: Virtual vs Pure Virtual

| Feature | Virtual Function | Pure Virtual Function |
|---|---|---|
| Definition in base | Has a body/definition | Has NO body; declared as =0 |
| Override in derived | Optional — can be overridden | MUST be overridden |
| Makes class abstract? | No | Yes |
| Create base object? | Yes, can create object | No, cannot create object |

## 5.8 Important Points

**Must Remember**

- Virtual functions enable RUNTIME POLYMORPHISM.
- Pure virtual function syntax: virtual returnType funcName() = 0;
- Abstract class CANNOT be instantiated (no objects).
- Virtual functions use a vtable (virtual function table) internally.
- Destructor should be declared virtual in base class when using pointers.
- Abstract class CAN have constructors and concrete functions.

## 5.9 Exam Questions

### Questions to Prepare

- What is a virtual function? Explain with example. (4 marks)
- What is a pure virtual function? How is it different from virtual function? (4 marks)
- What is an abstract class? Can you create objects of abstract class? (4 marks)
- Explain dynamic binding with example. (4 marks)
- Write a program to demonstrate polymorphism using virtual functions. (6 marks)

# TOPIC 6: Polymorphism — Function Overloading & Operator Overloading

## 6.1 Simple Explanation

Polymorphism means 'many forms'. The same function name or operator can behave differently based on context. Example: The '+' operator adds numbers (5+3=8) but concatenates strings ('Hello'+'World'='HelloWorld'). Same operator, different behavior.

## 6.2 Types of Polymorphism

| Type | Also Called | When Resolved | Achieved By |
|---|---|---|---|
| Compile-time | Static / Early binding | At compile time | Function overloading, Operator overloading |
| Runtime | Dynamic / Late binding | At runtime | Virtual functions |

## 6.3 Function Overloading

Multiple functions with the SAME NAME but DIFFERENT parameters (number, type, or order). Compiler selects correct function based on arguments passed.

```
class Calculator {
public:
    int add(int a, int b) { return a + b; }
    float add(float a, float b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
};

Calculator c;
c.add(5, 3);           // Calls first function (int version)
c.add(2.5f, 3.7f);     // Calls second function (float version)
c.add(1, 2, 3);        // Calls third function (3 params)
```

IMPORTANT: Functions cannot be differentiated by return type ALONE. They must differ in parameter list.

## 6.4 Operator Overloading

Redefining the behavior of existing operators for user-defined types (objects). Uses the 'operator' keyword.

```
class Complex {
public:
    float real, imag;
    Complex(float r, float i) { real = r; imag = i; }

    Complex operator+(Complex c) {   // Overloading + operator
        Complex temp(0, 0);
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;
    }

    void display() {
        cout << real << "+" << imag << "i";
    }
};
```

```
Complex c1(3, 2), c2(1, 4);
Complex c3 = c1 + c2;   // Uses overloaded + operator
c3.display();   // Output: 4+6i
```

## 6.5 Operators That CAN Be Overloaded

- Arithmetic: + - * / %
- Relational: == != < > <= >=
- Logical: && || !
- Unary: ++ -- - (negation)
- Assignment: = += -= *=
- Input/Output: << >>

## 6.6 Operators That CANNOT Be Overloaded

- Scope resolution operator (::)
- Member access operator (.)
- Pointer to member operator (->*)
- Ternary/conditional operator (?:)
- sizeof and typeid

## 6.7 Key Definitions

| Term | Definition |
|---|---|
| Polymorphism | Ability of a function/operator to behave differently based on context or object type. |
| Function Overloading | Defining multiple functions with same name but different parameter list in same scope. |
| Operator Overloading | Giving new definition to existing operators for user-defined objects using 'operator' keyword. |
| Compile-time Polymorphism | Polymorphism resolved at compile time (static/early binding). |
| Runtime Polymorphism | Polymorphism resolved at runtime via virtual functions (dynamic/late binding). |

## 6.8 Important Points

**Must Remember**

• Function overloading: same name, DIFFERENT parameters.
• Functions cannot differ ONLY in return type.
• Operator overloading uses 'operator' keyword.
• At least ONE operand must be a user-defined type.
• Overloaded operator maintains original precedence and associativity.
• ::. ->* ?: sizeof cannot be overloaded.
• Function overloading = compile-time polymorphism.
• Virtual functions = runtime polymorphism.

## 6.9 Exam Questions

### Questions to Prepare

- What is polymorphism? What are its types? (4 marks)
- Explain function overloading with example. (4 marks)
- What is operator overloading? Write a program to overload '+' for complex numbers. (6 marks)
- Which operators cannot be overloaded in C++? (2 marks)
- Differentiate between compile-time and runtime polymorphism. (4 marks)

# TOPIC 7: Templates — Class Template & Function Template

## 7.1 Simple Explanation

Templates allow you to write GENERIC code that works with any data type. Instead of writing separate functions for int, float, double — write ONE template and C++ automatically creates the correct version for each data type.

Think of a template as a cookie cutter: same shape, any dough (any data type).

## 7.2 Function Templates

A function template creates a single function that works with multiple data types.

```
template <typename T>    // T is a placeholder for any data type
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << findMax(10, 20);        // Works with int, T=int
    cout << findMax(3.5, 2.7);     // Works with float, T=float
    cout << findMax('A', 'Z');      // Works with char, T=char
}
```

- 'template <typename T>' declares T as a placeholder type.
- Also written as 'template <class T>' — both are equivalent.
- T can be replaced with any data type when calling the function.

## 7.3 Class Templates

A class template creates a generic class that can work with any data type.

```
template <typename T>
class Stack {
private:
    T arr[100];
    int top;
public:
    Stack() { top = -1; }
    void push(T val) { arr[++top] = val; }
    T pop() { return arr[top--]; }
    bool isEmpty() { return top == -1; }
};

Stack<int>    s1;   // Stack of integers
Stack<float>  s2;   // Stack of floats
Stack<string> s3;   // Stack of strings
```

## 7.4 Key Definitions

| Term | Definition |
|---|---|
| Template | C++ feature that allows writing generic code that works with any data type. |
| Function Template | Blueprint for a function that can work with multiple data types. |
| Class Template | Blueprint for a class that can work with multiple data types. |

| | |
|---|---|
| Template Parameter | The placeholder (usually T) used in the template to represent any data type. |
| Template Instantiation | The process of creating a specific version of template for a given data type. |

## 7.5 Function Template vs Class Template

| Feature | Function Template | Class Template |
|---|---|---|
| Purpose | Generic function | Generic class |
| Keyword | template <typename T> | template <typename T> |
| Usage | Call function: findMax(5,3) | Create object: Stack<int> s; |
| Scope | Function level | Class level |

## 7.6 Important Points

**Must Remember**

- Template keyword: template <typename T> or template <class T>.
- Templates support code reusability without data type duplication.
- STL (Standard Template Library) is entirely built on templates.
- Multiple type parameters are possible: template <typename T, typename U>.
- Templates are resolved at COMPILE TIME (not runtime).
- Template specialization allows different behavior for specific types.

## 7.7 Exam Questions

**Questions to Prepare**

- What is a template in C++? Why is it used? (4 marks)
- Write a function template to find maximum of two values. (4 marks)
- Explain class template with example. (4 marks)
- Differentiate between function template and class template. (4 marks)

# TOPIC 8: Exception Handling

## 8.1 Simple Explanation

Errors during program execution (division by zero, file not found, invalid input) are called EXCEPTIONS. Exception handling allows the program to detect errors, handle them gracefully, and continue running instead of crashing. C++ uses three keywords: TRY, CATCH, THROW.

## 8.2 try-throw-catch Mechanism

| Keyword | Role |
|---|---|
| try | Block of code where exception might occur is written here. |
| throw | Used to signal/raise an exception when an error occurs. |
| catch | Block that catches and handles the exception thrown by try block. |

```
int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    try {
        if (b == 0)
            throw "Division by zero!";   // throw exception
        cout << "Result: " << a / b;
    }
    catch (const char* msg) {    // catch the exception
        cout << "Error: " << msg;
    }

    cout << "\nProgram continues normally...";
    return 0;
}
```

## 8.3 Multiple Catch Blocks

```
try {
    int x = -1;
    if (x < 0)   throw x;          // throws int
    if (x == 0)  throw "zero";     // throws string
}
catch (int e) {
    cout << "Integer exception: " << e;
}
catch (const char* e) {
    cout << "String exception: " << e;
}
catch (...) {      // catch-all: catches any remaining exception
    cout << "Unknown exception";
}
```

## 8.4 Rethrowing an Exception

```
try {
    try {
        throw "error occurred";
```

```
    }
    catch (const char* e) {
        cout << "Inner catch: handling...";
        throw;   // rethrow the same exception to outer catch
    }
}
catch (const char* e) {
    cout << "Outer catch: " << e;
}
```

## 8.5 Key Definitions

| Term | Definition |
|---|---|
| Exception | An unexpected event or error that occurs during program execution. |
| Exception Handling | Mechanism to detect and handle runtime errors gracefully to prevent crash. |
| try block | Block of code that may generate an exception. |
| throw | Statement used to signal/raise an exception. |
| catch block | Block of code that handles the thrown exception. |
| catch(...) | Catch-all handler that catches any type of exception. |

## 8.6 Important Points

**Must Remember**

- try block MUST be followed by one or more catch blocks.
- throw can throw any type: int, float, string, object.
- catch(...) catches all exceptions — always put it LAST.
- If exception is NOT caught, program terminates with error.
- Exception handling does NOT prevent exceptions — it HANDLES them.
- Standard exceptions: exception, runtime_error, logic_error (in <stdexcept>).

## 8.7 Exam Questions

**Questions to Prepare**

- What is exception handling? What are its advantages? (4 marks)
- Explain try, catch, and throw with example. (6 marks)
- Write a program to handle divide by zero exception. (4 marks)
- Explain multiple catch blocks with example. (4 marks)

# TOPIC 9: Stream Computation — Console I/O & File Handling

## 9.1 Simple Explanation

C++ uses STREAMS for input/output. A stream is like a pipe through which data flows. Console I/O handles keyboard and screen. File I/O handles reading and writing files on disk.

## 9.2 Console I/O Streams

| Stream | Object | Purpose |
|---|---|---|
| Input Stream | cin | Read data from keyboard |
| Output Stream | cout | Write data to screen |
| Error Stream | cerr | Show error messages (unbuffered) |
| Log Stream | clog | Show log messages (buffered) |

## 9.3 I/O Manipulators (#include <iomanip>)

```
#include <iomanip>
cout << setw(10) << "Name";          // Set field width to 10
cout << setprecision(2) << 3.14159;   // 2 decimal places: 3.14
cout << fixed << 3.14159;             // Fixed: 3.141590
cout << left  << "Hello";            // Left align
cout << right << "Hello";            // Right align
cout << setfill('*') << setw(10) << "Hi"; // Fill with *
```

## 9.4 File Handling

File handling allows programs to read from and write to files permanently on disk.

| Class | Purpose | Header |
|---|---|---|
| ofstream | Writing to file (output file stream) | <fstream> |
| ifstream | Reading from file (input file stream) | <fstream> |
| fstream | Both reading AND writing | <fstream> |

## 9.5 Writing to a File

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ofstream myFile("student.txt");    // Open/create file for writing
    if (myFile.is_open()) {
        myFile << "Name: Rahul" << endl;
        myFile << "Age: 20" << endl;
        myFile.close();                // ALWAYS close after use
        cout << "File written successfully!";
    } else {
        cout << "Cannot open file!";
    }
    return 0;
```

```
}
```

## 9.6 Reading from a File

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream myFile("student.txt");    // Open file for reading
    string line;
    if (myFile.is_open()) {
        while (getline(myFile, line)) {  // Read line by line
            cout << line << endl;
        }
        myFile.close();
    } else {
        cout << "File not found!";
    }
    return 0;
}
```

## 9.7 File Opening Modes

| Mode | Description |
|---|---|
| ios::in | Open for reading |
| ios::out | Open for writing (creates file if not exists, overwrites) |
| ios::app | Append — write to end of file (existing content preserved) |
| ios::ate | Open and move to end of file |
| ios::trunc | Truncate — erase file contents on open |
| ios::binary | Open in binary mode (for images, executables) |

## 9.8 Important Points

**Must Remember**

- Always close file after use: myFile.close()
- Check if file opened successfully: if(myFile.is_open())
- ios::app adds to end; ios::out overwrites from beginning.
- #include <fstream> is required for file handling.
- getline(myFile, str) reads entire line including spaces.
- Binary files store data in binary format; text files in ASCII.

## 9.9 Exam Questions

**Questions to Prepare**

- What is file handling? Explain file opening modes. (4 marks)
- Write a program to write data to a file and read it back. (6 marks)

- Differentiate between ofstream, ifstream, and fstream. (4 marks)
- What are I/O manipulators? Give examples. (4 marks)

# QUICK REVISION SUMMARY — Subject 1: OOP in C++

| Topic | Key Points to Remember |
|---|---|
| OOP Concepts | 4 Pillars: Encapsulation, Abstraction, Inheritance, Polymorphism. Class=blueprint, Object=instance. |
| Constructor | Same name as class, no return type, auto-called. Types: Default, Parameterized, Copy, Overloaded. |
| Destructor | ~ClassName(), no params, only one, called when object destroyed. |
| Friend Function | Non-member with private access. Declared with 'friend', no 'this' pointer. Not mutual, not inherited. |
| Inheritance | 5 types. Private members never inherited. Virtual base class solves diamond problem. |
| Virtual Function | Enables runtime polymorphism. Pure virtual (=0) makes class abstract. |
| Abstract Class | Has pure virtual function. Cannot instantiate. Forces derived to implement all pure virtual functions. |
| Function Overloading | Same name, different parameters. Compile-time polymorphism. |
| Operator Overloading | 'operator' keyword. :: . ?: sizeof cannot be overloaded. |
| Templates | template<typename T>. Generic programming. Function & Class templates. STL uses templates. |
| Exception Handling | try-throw-catch. catch(...) catches all. Handles runtime errors gracefully. |
| File Handling | ofstream (write), ifstream (read), fstream (both). Always close file. ios:: modes. |